**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ**
**Львівський національний університет імені Івана Франка**
**Факультет електроніки та комп'ютерних технологій**

# ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ № 4
з дисципліни "Методи та технології інженерії даних"
на тему "Інженерія даних для задачі класифікації"

Виконав
студент 3 курсу
групи ФЕП-33
Рак Богдан Ігорович

Перевірив
Асистент кафедри СП
Фостяк Маркіян Романович

**Львів – 2026**

**Мета:** набути практичних навичок підготовки та перетворення даних для задачі класифікації; сформувати різні набори ознак (усі ознаки, відібрані ознаки, головні компоненти); застосувати метод головних компонент (PCA); дослідити вплив інженерії даних на якість моделі машинного навчання за метриками Accuracy, Precision, Recall, F1-score та ROC-AUC.

**Про дані.** Використано набір UCI Automobile (205 автомобілів, 26 колонок), який досліджувався в лабораторній роботі № 3 (EDA). Для задачі класифікації цільову змінну price перетворено на бінарну: is_expensive = 1, якщо price > 18 000 $ (преміум-сегмент), інакше 0. Поріг фіксований (бізнес-поріг, а не квантиль за даними), тому через вибірку не виникає витоку інформації; він дає ≈18 % «дорогих» авто, тобто суттєвий дисбаланс класів. Колонку price вилучено з ознак як пряме джерело витоку.

**Модель.** Для всіх експериментів використано один класифікатор — LogisticRegression(max_iter=2000, class_weight='balanced', random_state=42), оскільки метою роботи є дослідження впливу підготовки та представлення ознак, а не порівняння алгоритмів.

**План роботи:** 1. Підготовка даних за результатами EDA → 2. Train/Test split → 3. Числові та категоріальні ознаки → 4. Дисбаланс класів → 5. Набори ознак A / B / C (PCA) → 6. Побудова моделей → 7. Оцінювання → 8. Порівняльний аналіз → 9. Фінальний pipeline → Висновки.

## Питання для допуску

**1. Що таке класифікація? Для чого будують ML-модель класифікації? Приклад.** Класифікація — задача навчання з учителем, у якій модель за ознаками об'єкта передбачає його **дискретний клас** (на відміну від регресії, де прогноз — число). Модель будують, щоб автоматично приймати/підтримувати рішення для нових об'єктів, для яких відповідь ще невідома. Приклад: за характеристиками автомобіля визначати, чи він належить до преміум-сегмента; фільтр спаму (спам / не спам); діагностика (хворий / здоровий).

**2. Для чого відділяти змінну target від набору даних?** Таргет — це те, що модель має *передбачити*. Якщо залишити його серед ознак `X`, модель просто «підгляне» відповідь (витік інформації) і показуватиме нереально високу якість, марну на нових даних. Тому `y` зберігаємо окремо, а `X` містить лише те, що буде відомо на момент прогнозу.

**3. Для чого розділяти набір даних на тренувальний та тестувальний?** Щоб оцінити **узагальнювальну здатність** моделі на даних, яких вона не бачила. Оцінка на тих самих даних, на яких модель навчалась, завищена (перенавчання). Тест-вибірка імітує «нові» дані й має залишатися незалежною до фінального оцінювання.

**4. Назвіть алгоритм класифікації.** Логістична регресія, k найближчих сусідів (KNN), дерево рішень, випадковий ліс, SVM, наївний Байєс, градієнтний бустинг. У роботі використовуємо **логістичну регресію**.

**5. Як обчислюють точність класифікації? Що таке confusion matrix?** Accuracy = (кількість правильних прогнозів) / (кількість усіх об'єктів) = (TP + TN) / (TP + TN + FP + FN). **Матриця помилок (confusion matrix)** — таблиця, у якій рядки — справжні класи, стовпці — передбачені:

| | **прогноз 0** | **прогноз 1** |
|---|---|---|
| **факт 0** | TN | FP |
| **факт 1** | FN | TP |

З неї виводяться Precision = TP/(TP+FP), Recall = TP/(TP+FN), F1 = 2·P·R/(P+R). За дисбалансу Accuracy оманлива (модель, що завжди каже «клас 0», має ≈82 % точності), тому дивимось також Precision/Recall/F1/ROC-AUC.

## Імпорти

```
import warnings
warnings.filterwarnings('ignore')


import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import chi2_contingency


from sklearn.model_selection import train_test_split, RepeatedStratifiedKFold, cross_validate
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder, FunctionTransformer
from sklearn.decomposition import PCA
from sklearn.feature_selection import SelectKBest, mutual_info_classif
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             roc_auc_score, confusion_matrix, ConfusionMatrixDisplay,
roc_curve)


sns.set_theme(style='whitegrid')
pd.set_option('display.max_columns', 60)
pd.set_option('display.width', 200)
RANDOM_STATE = 42
```

## Завдання 1. Підготовка даних за результатами EDA

Використовуємо висновки з лабораторної роботи 3. Розділяємо очищення на два види:

- **Безстанові операції** (виправлення типів, друкарських помилок, детерміновані перетворення) — не залежать від розподілу даних, тому безпечні до розбиття на train/test.
- **Операції, що «вчаться» з даних** (імпутація медіаною/модою, масштабування, кодування) — виконуємо **тільки у `Pipeline` після розбиття**, щоб не допустити data leakage.

### *1.1. Завантаження та перевірка типів*

```
raw = pd.read_csv('Automobile_data.csv')
print('Розмір:', raw.shape)
print('isna():', raw.isna().sum().sum(), '| значень "?":', (raw == '?').sum().sum())
raw.dtypes.value_counts()
```

```
Розмір: (205, 26)
isna(): 0 | значень "?": 59
```

```
str        16
int64       5
float64     5
Name: count, dtype: int64
```

### *1.2. Типи, пропуски, друкарські помилки (за висновками EDA)*

```
df = raw.replace('?', np.nan)

# колонки, числові за змістом (були text через '?')
to_numeric = ['normalized-losses', 'bore', 'stroke', 'horsepower', 'peak-rpm', 'price']
for c in to_numeric:
    df[c] = pd.to_numeric(df[c])

# друкарські помилки в назвах марок
df['make'] = df['make'].replace({'peugot': 'peugeot', 'alfa-romero': 'alfa-romeo'})

miss = df.isna().sum()
print('Пропуски після заміни "?" на NaN:')
display(pd.DataFrame({'missing': miss, '%': (miss / len(df) * 100).round(1)})[miss > 0].sort_values('missing', ascending=False))
```

```
Пропуски після заміни "?" на NaN:
```

| | missing | % |
|---|---|---|
| normalized-losses | 41 | 20.0 |
| bore | 4 | 2.0 |

| | missing | % |
|---|---|---|
| stroke | 4 | 2.0 |
| price | 4 | 2.0 |
| num-of-doors | 2 | 1.0 |
| horsepower | 2 | 1.0 |
| peak-rpm | 2 | 1.0 |

## *1.3. Пропуски: стратегія за висновками EDA*

| Ознака | Пропусків | Рішення | Де виконується |
|---|---|---|---|
| `price` | 4 | **видалити рядки** (з ціни будуємо таргет; імпутувати цільову не можна) | зараз (до split) |
| `normalized-losses` | 41 (20 %) | медіана + бінарний індикатор `nl_missing` (пропуски не випадкові — відсутні цілими марками) | індикатор — зараз (детермінований), медіана — у Pipeline |
| `bore`, `stroke` | по 4 | медіана (структурні пропуски роторних двигунів; сигнал про ротор несе `engine-type`) | Pipeline |
| `horsepower`, `peak-rpm` | по 2 | медіана | Pipeline |
| `num-of-doors` | 2 | мода | Pipeline |

```python
# 1) рядки без ціни — видаляємо
before = len(df)
df = df.dropna(subset=['price']).reset_index(drop=True)
print(f'Видалено рядків без price: {before - len(df)}; залишилось {len(df)}')


# 2) індикатор пропуску normalized-losses (детермінована ознака, не залежить від розподілу)
df['nl_missing'] = df['normalized-losses'].isna().astype(int)


# 3) num-of-cylinders записано словами, має природний порядок -> числова ординальна ознака
cyl_map = {'two': 2, 'three': 3, 'four': 4, 'five': 5, 'six': 6, 'eight': 8, 'twelve': 12}
df['num-of-cylinders'] = df['num-of-cylinders'].map(cyl_map)
df['num-of-cylinders'].value_counts().sort_index().to_frame('count').T
```

```
Видалено рядків без price: 4; залишилось 201
```

| num-of-cylinders | 2 | 3 | 4 | 5 | 6 | 8 | 12 |
|---|---|---|---|---|---|---|---|
| count | 4 | 1 | 157 | 10 | 24 | 4 | 1 |

### *1.4. Дублікати*

```
print('Повних дублікатів:', df.duplicated().sum())
feat_cols = [c for c in df.columns if c != 'price']
print('Рядків-«близнюків» (однакові всі ознаки, різна ціна):', df.duplicated(subset=feat_cols,
keep=False).sum())
```

```
Повних дублікатів: 0
Рядків-«близнюків» (однакові всі ознаки, різна ціна): 14
```

Повних дублікатів немає — видаляти нічого. «Близнюки» (різні комплектації однієї моделі) залишаємо: це різні об'єкти, і вони майже завжди належать до одного класу цін. Ризик витоку через них враховуємо в обговоренні (див. Завдання 9).

### *1.5. Некоректні та аномальні значення*

```
print('Логічні перевірки:')
print('  highway-mpg < city-mpg      :', (df['highway-mpg'] < df['city-mpg']).sum())
meas = ['normalized-losses', 'wheel-base', 'length', 'width', 'height', 'curb-weight',
'engine-size', 'bore', 'stroke',
        'compression-ratio', 'horsepower', 'peak-rpm', 'city-mpg', 'highway-mpg', 'price']
print('  значень <= 0 (вимірювання)   :', (df[meas] <= 0).sum().sum())
print('  symboling поза -3..3         :', (~df['symboling'].between(-3, 3)).sum())

# фізична узгодженість: engine-size ~ pi/4 * bore^2 * stroke * cylinders
implied = np.pi / 4 * df['bore']**2 * df['stroke'] * df['num-of-cylinders']
ratio = implied / df['engine-size']
bad = (ratio < 0.8) | (ratio > 1.2)
print(f'  фізично неузгоджених рядків  : {bad.sum()} з {ratio.notna().sum()}')
df.loc[bad, ['make', 'num-of-cylinders', 'bore', 'stroke', 'engine-
size']].assign(ratio=ratio[bad].round(2))
```

```
Логічні перевірки:
  highway-mpg < city-mpg      : 0
  значень <= 0 (вимірювання)   : 0
  symboling поза -3..3         : 0
  фізично неузгоджених рядків  : 6 з 197
```

| | make | num-of-cylinders | bore | stroke | engine-size | ratio |
|---|---|---|---|---|---|---|
| 0 | alfa-romeo | 4 | 3.47 | 2.68 | 130 | 0.78 |
| 1 | alfa-romeo | 4 | 3.47 | 2.68 | 130 | 0.78 |
| 2 | alfa-romeo | 6 | 2.68 | 3.47 | 152 | 0.77 |
| 108 | peugeot | 4 | 3.46 | 2.19 | 120 | 0.69 |
| 110 | peugeot | 4 | 3.46 | 2.19 | 120 | 0.69 |
| 130 | saab | 4 | 2.54 | 2.07 | 121 | 0.35 |

**Рішення щодо аномалій.**

- Викиди `price`, `engine-size`, `horsepower`, `compression-ratio` — це **реальні** підгрупи (люкс-марки, дизелі), а не помилки (висновок EDA). Не видаляємо: саме вони визначають клас «дорогий».

- Шість фізично неузгоджених рядків (Saab, 2×Peugeot, 3×Alfa Romeo) — це ≈3 % даних (сьомий проблемний рядок із EDA, Porsche, вже видалено разом із порожньою ціною), а джерела для перевірки немає. Залишаємо як є: логістична регресія з регуляризацією до таких поодиноких значень малочутлива, а видалення зменшило б і без того малу вибірку.
- Різні масштаби ознак і правобічна асиметрія враховуються масштабуванням у Завданні 3.

### *1.6. Цільова змінна y, початковий набір ознак X, видалення небажаних ознак*

```
PRICE_THRESHOLD = 18_000
y = (df['price'] > PRICE_THRESHOLD).astype(int).rename('is_expensive')

# price — джерело таргета (пряма ознака витоку) -> прибираємо з X
X = df.drop(columns=['price'])

# ідентифікаторів і службових полів у наборі немає (EDA, п. 5.2)
print('X:', X.shape, '| y:', y.shape)
print('Колонки X:', list(X.columns))
```

```
X: (201, 26) | y: (201,)
Колонки X: ['symboling', 'normalized-losses', 'make', 'fuel-type', 'aspiration', 'num-of-doors',
'body-style', 'drive-wheels', 'engine-location', 'wheel-base', 'length', 'width', 'height',
'curb-weight', 'engine-type', 'num-of-cylinders', 'engine-size', 'fuel-system', 'bore',
'stroke', 'compression-ratio', 'horsepower', 'peak-rpm', 'city-mpg', 'highway-mpg',
'nl_missing']
```

**Що виключено з ознак:**

- `price` — на ній побудований таргет (**витік інформації** про ціль);
- ідентифікаторів / службових полів у наборі немає (перевірено в EDA);
- `normalized-losses` — **залишаємо** (не витік: це страхова статистика, відома до оцінки ціни), але з індикатором пропусків;
- `symboling` — страховий рейтинг, теж не залежить від ціни напряму.

Підсумок: **початкова кількість ознак — 26 колонок у `X`** (25 вихідних без `price` + `nl_missing`).

## Завдання 2. Навчальна та тестова вибірки

Співвідношення 80 / 20, `stratify=y` — щоб частка «дорогих» авто була однаковою в train і test.

```python
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=RANDOM_STATE)

print('train:', X_train.shape, '| test:', X_test.shape)
pd.DataFrame({
    'train': y_train.value_counts(normalize=True).round(3),
    'test': y_test.value_counts(normalize=True).round(3),
    'всього': y.value_counts(normalize=True).round(3)}).rename_axis('class')
```

```
train: (160, 26) | test: (41, 26)
```

| | train | test | всього |
|---|---|---|---|
| 0 | 0.819 | 0.829 | 0.821 |
| 1 | 0.181 | 0.171 | 0.179 |

Частки класів майже однакові у всіх трьох вибірках — стратифікація спрацювала. Усі подальші «навчені» кроки (імпутація, масштабування, кодування, відбір ознак, PCA, балансування) налаштовуємо **лише на `X_train`**; `X_test` використовуємо тільки для `transform`/`predict` у фінальній оцінці.

## Завдання 3. Числові та категоріальні ознаки

```python
categorical = ['make', 'fuel-type', 'aspiration', 'num-of-doors', 'body-style',
               'drive-wheels', 'engine-location', 'engine-type', 'fuel-system']
numeric = [c for c in X.columns if c not in categorical]

print(f'Числові ({len(numeric)}):', numeric)
print(f'\nКатегоріальні ({len(categorical)}):', categorical)
```

```
Числові (17): ['symboling', 'normalized-losses', 'wheel-base', 'length', 'width', 'height',
'curb-weight', 'num-of-cylinders', 'engine-size', 'bore', 'stroke', 'compression-ratio',
'horsepower', 'peak-rpm', 'city-mpg', 'highway-mpg', 'nl_missing']


Категоріальні (9): ['make', 'fuel-type', 'aspiration', 'num-of-doors', 'body-style', 'drive-
wheels', 'engine-location', 'engine-type', 'fuel-system']
```

### *Обґрунтування способів підготовки*

| Група | Ознаки | Спосіб | Чому |
|---|---|---|---|
| Числові (неперервні) | `wheel-base`, `length`, `width`, `height`, `curb-weight`, `engine-size`, `bore`, `stroke`, `compression-ratio`, `horsepower`, `peak-rpm`, `city-mpg`, `highway-mpg`, `normalized-losses` | `SimpleImputer(median)` → `StandardScaler` | Медіана стійка до викидів і асиметрії (EDA: skew до 2.6). Логістична регресія з L2-регуляризацією та PCA чутливі до масштабу (`peak-rpm` ~5000 проти `bore` ~3) |
| Числові (ординальні/бінарні) | `symboling`, `num-of-cylinders`, `nl_missing` | так само | Мають природний порядок / лише 0-1; числове подання зберігає порядок і не роздуває розмірність |
| Категоріальні | `make`, `fuel-type`, `aspiration`, `num-of-doors`, `body-style`, `drive-wheels`, `engine-location`, `engine-type`, `fuel-system` | `SimpleImputer(most_frequent)` → `OneHotEncoder(handle_unknown='infrequent_if_exist', min_frequency=5)` | Категорії без порядку -> OHE. Рідкісні категорії (< 5 записів: `dohcv`, `mfi`, `spfi`, `mercury` тощо — висновок EDA) об'єднуються в одну «infrequent», а невідомі в test не ламають модель |

Усе зібрано в `ColumnTransformer`, який разом із класифікатором входить у `Pipeline` — тому `fit` відбувається тільки на train.

```
def make_preprocessor(num_cols, cat_cols, pca_components=None):
    steps = [('imputer', SimpleImputer(strategy='median')), ('scaler', StandardScaler())]
    if pca_components is not None:
        steps.append(('pca', PCA(n_components=pca_components, random_state=RANDOM_STATE)))
    transformers = []
    if num_cols:
        transformers.append(('num', Pipeline(steps), num_cols))
    if cat_cols:
        transformers.append(('cat', Pipeline([
            ('imputer', SimpleImputer(strategy='most_frequent')),
            ('onehot', OneHotEncoder(handle_unknown='infrequent_if_exist',
                                     min_frequency=5, sparse_output=False))]), cat_cols))
    return ColumnTransformer(transformers)


def make_classifier(class_weight='balanced'):
    # ОДНА й та сама модель з одними параметрами для всіх експериментів
    return LogisticRegression(max_iter=2000, class_weight=class_weight,
random_state=RANDOM_STATE)


all_cols = numeric + categorical
prep_all = make_preprocessor(numeric, categorical).fit(X_train[all_cols])
n_encoded = len(prep_all.get_feature_names_out())
print(f'Було колонок: {len(all_cols)}  ->  після кодування: {n_encoded} ознак')
print('Структура кодованих ознак:',

pd.Series(prep_all.get_feature_names_out()).str.split('__').str[0].value_counts().to_dict())
```

```
Було колонок: 26  ->  після кодування: 60 ознак
Структура кодованих ознак: {'cat': 43, 'num': 17}
```

# Завдання 4. Дисбаланс цільової змінної

```
cnt = pd.DataFrame({
    'кількість': y.value_counts(),
    'частка, %': (y.value_counts(normalize=True) * 100).round(1)})
cnt.index = cnt.index.map({0: '0 — не дорогий (≤ 18 000)', 1: '1 — дорогий (> 18 000)'})
display(cnt)


ir = y.value_counts()[0] / y.value_counts()[1]
print(f'Співвідношення мажоритарного до міноритарного класу: {ir:.1f} : 1')


fig, ax = plt.subplots(1, 3, figsize=(13, 3.5))
for a, (name, s) in zip(ax, [('Усі дані', y), ('Train', y_train), ('Test', y_test)]):
    s.value_counts().sort_index().plot.bar(ax=a, color=['steelblue', 'darkorange'])
    a.set_title(f'{name} (n={len(s)})'); a.set_xlabel('is_expensive'); a.tick_params(axis='x',
rotation=0)
    for p in a.patches:
        a.annotate(int(p.get_height()), (p.get_x() + p.get_width() / 2, p.get_height()),
ha='center', va='bottom')
plt.tight_layout(); plt.show()
```

| | кількість | частка, % |
|---|---|---|
| 0 — не дорогий (≤ 18 000) | 165 | 82.1 |
| 1 — дорогий (> 18 000) | 36 | 17.9 |

```
Співвідношення мажоритарного до міноритарного класу: 4.6 : 1
```

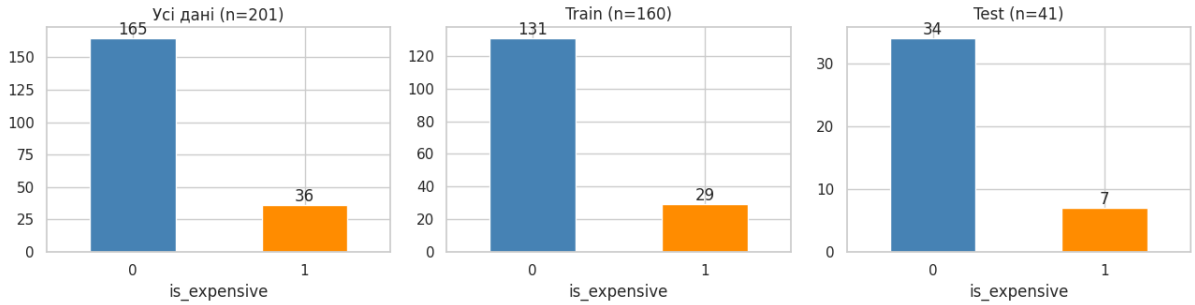



*Рисунок 1 — Розподіл класів цільової змінної (усі дані, train, test)*

**Висновок:** клас «дорогий» становить лише ≈18 % (≈4.6 : 1) — дисбаланс **суттєвий**. Наївна модель «завжди 0» мала б Accuracy ≈ 82 %, але Recall = 0 для цікавого нам класу.

**Обраний метод — `class_weight='balanced'`.** Чому саме він:

- об'єктів усього ≈160 у train (~29 «дорогих») — **Random Undersampling** викинув би більшість даних, **Random Oversampling** просто дублював би 29 рядків (ризик перенавчання), **SMOTE** синтезує «середні» авто між реальними, що для змішаних числових + OHE-ознак фізично сумнівно;
- `class_weight` не змінює дані, а збільшує штраф за помилки на міноритарному класі, тому працює лише на навчанні й **не торкається test**; не потребує додаткових бібліотек і не може спричинити витік.

Перевіримо ефект на train за допомогою повторної стратифікованої крос-валідації (test поки не чіпаємо).

```
cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=5, random_state=RANDOM_STATE)
scoring = {'accuracy': 'accuracy', 'precision': 'precision', 'recall': 'recall', 'f1': 'f1',
'roc_auc': 'roc_auc'}


def cv_report(pipe, cols, name):
    r = cross_validate(pipe, X_train[cols], y_train, cv=cv, scoring=scoring, n_jobs=-1)
    return pd.Series({m: r[f'test_{m}'].mean() for m in scoring}, name=name)


cmp_bal = pd.DataFrame([
    cv_report(Pipeline([('prep', make_preprocessor(numeric, categorical)), ('clf',
make_classifier(None))]), all_cols, 'без балансування'),
    cv_report(Pipeline([('prep', make_preprocessor(numeric, categorical)), ('clf',
make_classifier('balanced'))]), all_cols, "class_weight='balanced'")])
cmp_bal.round(3)
```

| | accuracy | precision | recall | f1 | roc_auc |
|---|---|---|---|---|---|
| без балансування | 0.919 | 0.845 | 0.700 | 0.739 | 0.937 |
| class_weight='balanced' | 0.891 | 0.679 | 0.845 | 0.742 | 0.938 |

**Висновок.** Балансування змінило *баланс помилок*, а не загальну якість ранжування: Recall міноритарного класу виріс із **0.70 до 0.85** (модель рідше «пропускає» дорогі авто), ціною падіння Precision (0.85 → 0.68) та Accuracy (0.92 → 0.89). F1 (0.74 в обох) і ROC-AUC (0.94 в обох) практично не змінились. Падіння Accuracy тут не означає гіршу модель — без балансування частину точності давала сама більшість класу «0». Для задачі «знайти преміум-авто» пропуск дорогого авто небажаний, тому `class_weight='balanced'` використовуємо в усіх моделях. Test-вибірка не балансувалась.

## Завдання 5. Формування наборів ознак

### Набір A — усі ознаки

Усі 26 колонок після очищення; після кодування — `prep_all` (див. вище). Нічого не відкидаємо.

```
set_A = all_cols
print('Набір A:', len(set_A), 'колонок ->', n_encoded, 'кодованих ознак')
```

```
Набір A: 26 колонок -> 60 кодованих ознак
```

### Набір B — відібрані ознаки

Відбір виконуємо **тільки за train** (щоб не «підглядати» в test). Комбінуємо три підходи, обґрунтовані в EDA:

1. **Кореляційний аналіз** (числові ознаки): зв'язок з таргетом (точково-бісеріальна кореляція = Пірсон з 0/1).
2. **Міра зв'язку для категоріальних ознак** — коефіцієнт Крамера $V$ ($\chi^2$-залежність з таргетом).
3. **Боротьба з мультиколінеарністю**: з пари ознак із $|r| > 0.85$ залишаємо ту, що сильніше корелює з таргетом (EDA: `city-mpg`↔`highway-mpg` 0.97, габарити ↔ `curb-weight` 0.84–0.88).
4. **Feature Importance** (випадковий ліс) — як перехресна перевірка результату.

## 5.1. Числові ознаки

```
train_df = X_train.assign(target=y_train)
corr_t = train_df[numeric].corrwith(train_df['target']).rename('r з target')
corr_t = corr_t.reindex(corr_t.abs().sort_values(ascending=False).index)

plt.figure(figsize=(8, 5))
colors = ['firebrick' if v < 0 else 'steelblue' for v in corr_t]
corr_t.iloc[::-1].plot.barh(color=colors[::-1])
plt.axvline(0.4, color='gray', ls='--'); plt.axvline(-0.4, color='gray', ls='--')
plt.title('Кореляція числових ознак з таргетом (train)'); plt.tight_layout(); plt.show()
corr_t.round(3).to_frame()
```

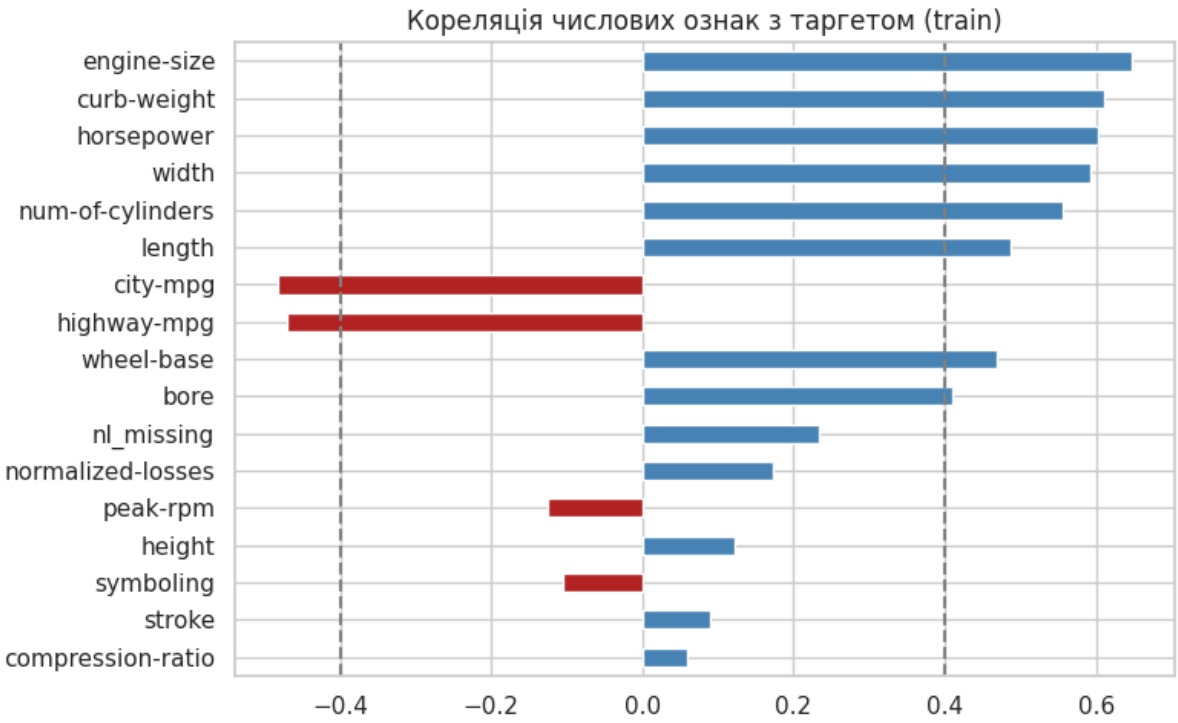



*Рисунок 2 — Кореляція числових ознак із цільовою змінною (train)*

| | r з target |
|---|---|
| engine-size | 0.647 |
| curb-weight | 0.611 |
| horsepower | 0.603 |
| width | 0.592 |
| num-of-cylinders | 0.555 |
| length | 0.486 |
| city-mpg | -0.482 |
| highway-mpg | -0.471 |
| wheel-base | 0.468 |
| bore | 0.411 |
| nl_missing | 0.233 |
| normalized-losses | 0.172 |
| peak-rpm | -0.125 |
| height | 0.122 |
| symboling | -0.105 |

| | r з target |
|---|---|
| stroke | 0.089 |
| compression-ratio | 0.059 |

```python
# Крок 1: поріг |r| >= 0.4
TH_R, TH_COLLIN = 0.4, 0.85
cand = corr_t[corr_t.abs() >= TH_R].index.tolist()

# Крок 2: «жадібне» прибирання мультиколінеарності (від найсильнішої до найслабшої)
corr_feat = X_train[numeric].corr().abs()
selected_num = []
dropped_collin = {}
for f in cand:                                    # cand вже відсортований за |r| з target
    clash = [s for s in selected_num if corr_feat.loc[f, s] > TH_COLLIN]
    if clash:
        dropped_collin[f] = (clash[0], round(corr_feat.loc[f, clash[0]], 2))
    else:
        selected_num.append(f)

print('Пройшли поріг |r| >= 0.4:', cand)
print('\nВідкинуті як мультиколінеарні (ознака: (з ким, |r|)):')
for k, v in dropped_collin.items(): print(f'  {k:12s} <- дублює {v[0]} (|r|={v[1]})')
print('\nЗалишені числові:', selected_num)
print('Відкинуті через слабкий зв\'язок з target (|r| < 0.4):', [c for c in numeric if c not
in cand])
```

```
Пройшли поріг |r| >= 0.4: ['engine-size', 'curb-weight', 'horsepower', 'width', 'num-of-
cylinders', 'length', 'city-mpg', 'highway-mpg', 'wheel-base', 'bore']


Відкинуті як мультиколінеарні (ознака: (з ким, |r|)):
  curb-weight  <- дублює engine-size (|r|=0.87)
  length       <- дублює width (|r|=0.85)
  highway-mpg  <- дублює city-mpg (|r|=0.97)


Залишені числові: ['engine-size', 'horsepower', 'width', 'num-of-cylinders', 'city-mpg', 'wheel-
base', 'bore']
Відкинуті через слабкий зв'язок з target (|r| < 0.4): ['symboling', 'normalized-losses',
'height', 'stroke', 'compression-ratio', 'peak-rpm', 'nl_missing']
```

## *5.2. Категоріальні ознаки*

```python
def cramers_v(a, b):
    t = pd.crosstab(a, b)
    chi2 = chi2_contingency(t)[0]
    return np.sqrt(chi2 / (t.values.sum() * (min(t.shape) - 1)))

cv_cat = pd.Series({c: cramers_v(X_train[c].fillna('NA'), y_train) for c in categorical},
                   name="Cramér's V").sort_values(ascending=False)
plt.figure(figsize=(7, 3.5)); cv_cat.iloc[::-1].plot.barh(color='teal'); plt.axvline(0.3,
color='gray', ls='--')
plt.title("Зв'язок категоріальних ознак з таргетом (train)"); plt.tight_layout(); plt.show()

TH_V = 0.3
selected_cat = cv_cat[cv_cat >= TH_V].index.tolist()
print('Залишені категоріальні (V >= 0.3):', selected_cat)
print('Відкинуті:', [c for c in categorical if c not in selected_cat])
cv_cat.round(3).to_frame()
```




*Рисунок 3 — Зв'язок категоріальних ознак із цільовою змінною: коефіцієнт Крамера V (train)*

```
Залишені категоріальні (V >= 0.3): ['make', 'drive-wheels', 'fuel-system', 'body-style']
Відкинуті: ['fuel-type', 'aspiration', 'num-of-doors', 'engine-location', 'engine-type']
```

| | **Cramér's V** |
|---|---|
| make | 0.775 |
| drive-wheels | 0.499 |
| fuel-system | 0.416 |
| body-style | 0.315 |
| engine-type | 0.283 |
| aspiration | 0.243 |
| engine-location | 0.166 |
| fuel-type | 0.075 |
| num-of-doors | 0.056 |

### *5.3. Перехресна перевірка: Feature Importance (RandomForest, train)*

```
rf = Pipeline([('prep', make_preprocessor(numeric, categorical)),
               ('rf', RandomForestClassifier(500, class_weight='balanced',
random_state=RANDOM_STATE))]).fit(X_train[all_cols], y_train)
names = rf['prep'].get_feature_names_out()
imp = pd.Series(rf['rf'].feature_importances_, index=names)

# агрегуємо важливість OHE-стовпців назад до вихідних колонок
orig = imp.groupby(lambda n: next((c for c in all_cols if n.split('__', 1)[1].startswith(c)),
n)).sum().sort_values(ascending=False)
plt.figure(figsize=(8, 6)); orig.iloc[::-1].plot.barh(color='seagreen')
plt.title('Feature importance (RandomForest), агреговано за вихідними колонками');
plt.tight_layout(); plt.show()
print('Топ-10:', orig.head(10).round(3).to_dict())
```

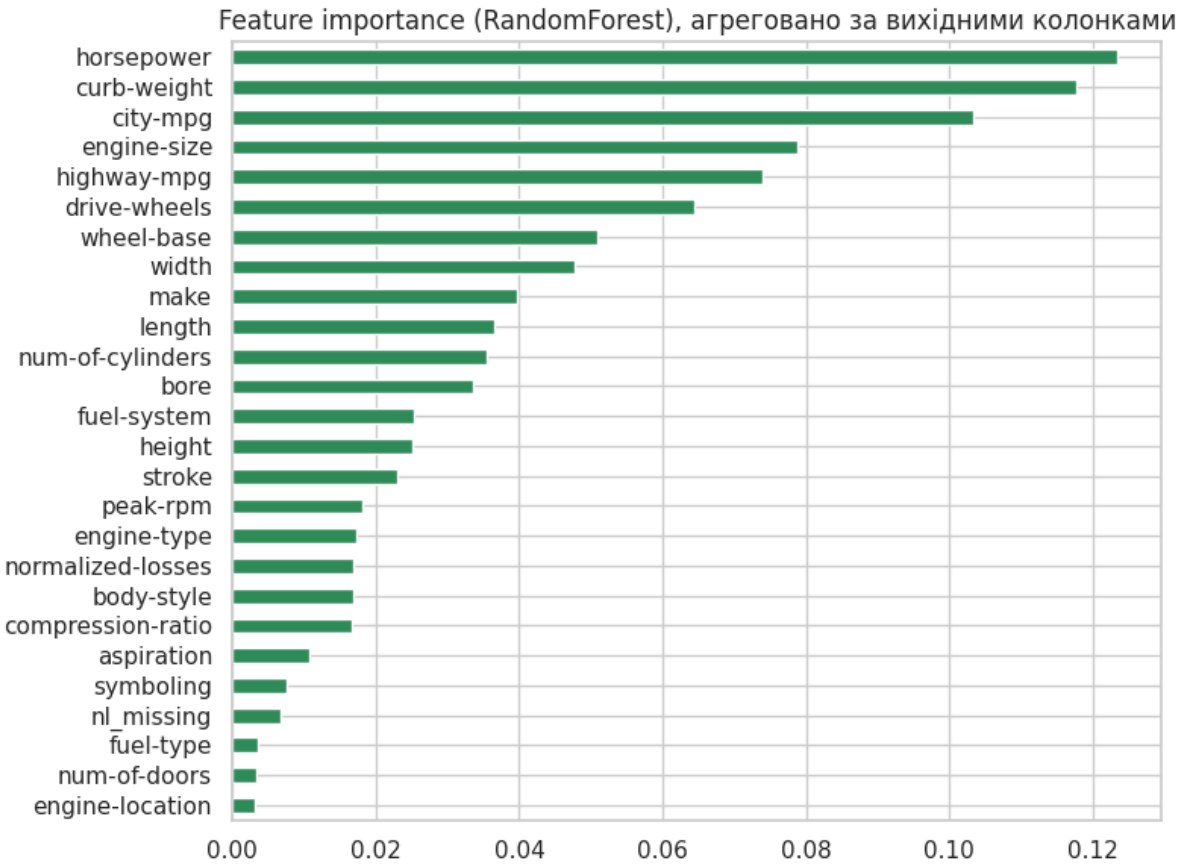



*Рисунок 4 — Feature importance (RandomForest), агреговано за вихідними колонками*

```
Топ-10: {'horsepower': 0.123, 'curb-weight': 0.118, 'city-mpg': 0.103, 'engine-size': 0.079,
'highway-mpg': 0.074, 'drive-wheels': 0.065, 'wheel-base': 0.051, 'width': 0.048, 'make': 0.04,
'length': 0.037}
```

```python
set_B = selected_num + selected_cat
dropped_B = [c for c in all_cols if c not in set_B]
print(f'Набір B: {len(set_B)} колонок з {len(all_cols)}')
print('  числові :', selected_num)
print('  категор.:', selected_cat)
print(f'Вилучено ({len(dropped_B)}):', dropped_B)
print('\nВажливість за RandomForest для вилучених колонок (сума, %):',
      (orig.reindex(dropped_B).fillna(0).sum() * 100).round(1), '| для залишених:',
(orig.reindex(set_B).sum() * 100).round(1))
```

```
Набір B: 11 колонок з 26
  числові : ['engine-size', 'horsepower', 'width', 'num-of-cylinders', 'city-mpg', 'wheel-base',
'bore']
  категор.: ['make', 'drive-wheels', 'fuel-system', 'body-style']
Вилучено (15): ['symboling', 'normalized-losses', 'length', 'height', 'curb-weight', 'stroke',
'compression-ratio', 'peak-rpm', 'highway-mpg', 'nl_missing', 'fuel-type', 'aspiration', 'num-
of-doors', 'engine-location', 'engine-type']

Важливість за RandomForest для вилучених колонок (сума, %): 38.0 | для залишених: 62.0
```

### *Що залишено і що вилучено в наборі B (11 з 26 колонок)*

**Залишено (числові):** `engine-size`, `horsepower`, `width`, `num-of-cylinders`, `city-mpg`, `wheel-base`, `bore` — |r| з таргетом ≥ 0.4, по одному представнику з кожної групи взаємопов'язаних ознак (потужність/об'єм, габарити, економічність).

**Залишено (категоріальні):** `make` (V = 0.78 — марка майже визначає цінову категорію), `drive-wheels` (0.50; 32 із 36 дорогих авто — rwd), `fuel-system` (0.42), `body-style` (0.32).

**Вилучено:**

| Колонки | Причина |
|---|---|
| `curb-weight`, `length`, `highway-mpg` | **мультиколінеарні** дублікати залишених `engine-size` (r за модулем 0.87), `width` (0.85), `city-mpg` (0.97): майже не додають нової інформації |
| `symboling`, `normalized-losses`, `nl_missing`, `height`, `stroke`, `compression-ratio`, `peak-rpm` | слабкий зв'язок із таргетом (r за модулем < 0.4), у EDA — переважно «шумові» ознаки (`stroke` — навіть «сам по собі», максимальна r за модулем = 0.21) |
| `fuel-type`, `aspiration`, `num-of-doors`, `engine-type`, `engine-location` | слабкий зв'язок (V < 0.3); `engine-location` до того ж майже константа (98.5 % `front`), `fuel-type` — 90 % `gas` |

**Перехресна перевірка за Feature Importance.** Вилучені колонки містять ≈38 % сумарної важливості випадкового лісу, але переважно це `curb-weight` та `highway-mpg` — вони «ділять» важливість із залишеними колонками, які їх дублюють (за кореляцією 0.87 та 0.97), тож інформація не втрачається. Топ-ознаки RF (`horsepower`, `city-mpg`, `engine-size`, `drive-wheels`, `wheel-base`, `width`) майже всі потрапили в набір B.

### Набір C — PCA

Послідовність: імпутація → **стандартизація** → PCA, усе навчається на `X_train`. PCA застосовуємо до **підготовлених числових даних** (17 числових ознак); категоріальні (OHE) PCA не перетворюємо — бінарні індикатори після PCA втрачають інтерпретацію й змішуються з числовими, а розріджені 0/1-стовпці мають інший характер дисперсії. Тож числовий блок замінюється головними компонентами, а закодовані категорії додаються як є.

```
num_pipe = Pipeline([('imputer', SimpleImputer(strategy='median')), ('scaler',
StandardScaler())]).fit(X_train[numeric])
Z_train = num_pipe.transform(X_train[numeric])


pca_full = PCA(random_state=RANDOM_STATE).fit(Z_train)
evr = pca_full.explained_variance_ratio_
cum = np.cumsum(evr)
pca_table = pd.DataFrame({'PC': [f'PC{i+1}' for i in range(len(evr))],
                          'explained_variance_ratio': evr.round(4), 'накопичена':
cum.round(4)}).set_index('PC')
display(pca_table)


n90 = int(np.argmax(cum >= 0.90) + 1)
n95 = int(np.argmax(cum >= 0.95) + 1)
print(f'90 % дисперсії: {n90} компонент | 95 %: {n95} компонент (з {Z_train.shape[1]} числових
ознак)')


fig, ax = plt.subplots(1, 2, figsize=(13, 4))
ax[0].bar(range(1, len(evr) + 1), evr, color='steelblue');
ax[0].set_title('explained_variance_ratio_ (scree plot)')
ax[0].set_xlabel('компонента'); ax[0].set_ylabel('частка дисперсії')
ax[1].plot(range(1, len(cum) + 1), cum, marker='o')
ax[1].axhline(0.90, color='orange', ls='--', label='90 %'); ax[1].axhline(0.95, color='red',
ls='--', label='95 %')
ax[1].axvline(n95, color='red', ls=':'); ax[1].set_title('Накопичена пояснена дисперсія')
ax[1].set_xlabel('кількість компонент'); ax[1].legend()
plt.tight_layout(); plt.show()
```

| | explained_variance_ratio | накопичена |
|---|---|---|
| PC1 | 0.4342 | 0.4342 |
| PC2 | 0.1747 | 0.6089 |
| PC3 | 0.0844 | 0.6933 |
| PC4 | 0.0642 | 0.7575 |
| PC5 | 0.0582 | 0.8157 |
| PC6 | 0.0494 | 0.8651 |
| PC7 | 0.0361 | 0.9013 |
| PC8 | 0.0257 | 0.9270 |
| PC9 | 0.0210 | 0.9480 |
| PC10 | 0.0178 | 0.9657 |
| PC11 | 0.0133 | 0.9790 |
| PC12 | 0.0059 | 0.9849 |
| PC13 | 0.0055 | 0.9904 |
| PC14 | 0.0050 | 0.9954 |
| PC15 | 0.0027 | 0.9981 |

| | explained_variance_ratio | накопичена |
|---|---|---|
| PC16 | 0.0012 | 0.9993 |
| PC17 | 0.0007 | 1.0000 |

```
90 % дисперсії: 7 компонент | 95 %: 10 компонент (з 17 числових ознак)
```

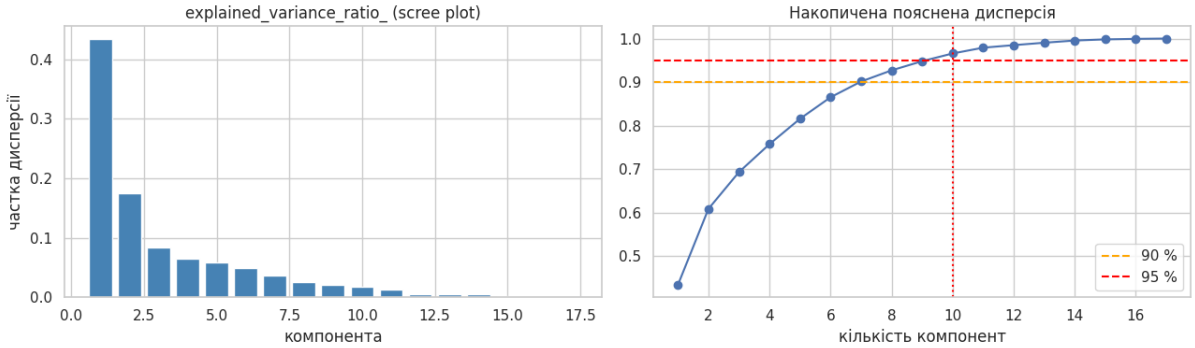



*Рисунок 5 — Пояснена дисперсія PCA: scree plot та накопичена дисперсія*

```python
# Навантаження перших компонент — що вони означають
loadings = pd.DataFrame(pca_full.components_[:3].T, index=numeric, columns=['PC1', 'PC2',
'PC3']).round(2)
loadings.sort_values('PC1', key=abs, ascending=False)
```

| | PC1 | PC2 | PC3 |
|---|---|---|---|
| curb-weight | 0.36 | -0.02 | 0.07 |
| engine-size | 0.33 | 0.05 | 0.19 |
| width | 0.33 | -0.06 | 0.06 |
| length | 0.33 | -0.13 | -0.05 |
| highway-mpg | -0.31 | -0.21 | 0.20 |
| city-mpg | -0.30 | -0.26 | 0.19 |
| horsepower | 0.30 | 0.27 | 0.00 |
| wheel-base | 0.29 | -0.25 | -0.06 |
| bore | 0.26 | -0.02 | -0.22 |
| num-of-cylinders | 0.24 | 0.08 | 0.16 |
| nl_missing | 0.15 | 0.05 | 0.25 |
| height | 0.12 | -0.41 | -0.27 |
| peak-rpm | -0.10 | 0.36 | -0.21 |
| stroke | 0.09 | -0.05 | 0.55 |
| symboling | -0.07 | 0.42 | 0.28 |
| normalized-losses | 0.04 | 0.34 | 0.21 |
| compression-ratio | 0.01 | -0.36 | 0.44 |

### *Висновки щодо PCA*

- **PC1 пояснює 43 %** дисперсії: «розмір і потужність» (додатні навантаження `curb-weight`, `engine-size`, `width`, `length`, `horsepower`; від'ємні —

`city/highway-mpg`). Це наслідок сильної мультиколінеарності, виявленої в EDA.

- **PC2 (17 %)** — контраст «низький автомобіль із високими оборотами й вищим страховим ризиком» (`symboling`, `peak-rpm`, `normalized-losses` проти `height`, `compression-ratio`); **PC3 (8 %)** — переважно `stroke` та `compression-ratio`.
- **90 % дисперсії — 7 компонент, 95 % — 10 компонент** (10 компонент пояснюють фактично **96.6 %**, бо 9-та дає лише 94.8 %). Беремо 10.
- **Скільки ознак замінено:** 17 числових → 10 головних компонент (−7); разом із 43 OHE-ознаками набір C має 53 ознаки (проти 60 у наборі A). Категоріальна частина не стискається, тому загальне зменшення розмірності невелике.

```
# Перетворення train і test побудованим PCA (n95 компонент) — лише transform для test!
pca_n = n95
prep_C = make_preprocessor(numeric, categorical, pca_components=pca_n).fit(X_train[all_cols])
Xtr_C, Xte_C = prep_C.transform(X_train[all_cols]), prep_C.transform(X_test[all_cols])
n_cat_enc = Xtr_C.shape[1] - pca_n
print(f'Набір C: {len(numeric)} числових ознак замінено на {pca_n} головних компонент '
      f'(пояснюють {cum[pca_n-1]*100:.1f} % дисперсії); + {n_cat_enc} OHE-ознак -> разом
{Xtr_C.shape[1]}')
print('train:', Xtr_C.shape, '| test:', Xte_C.shape)
```

```
Набір C: 17 числових ознак замінено на 10 головних компонент (пояснюють 96.6 % дисперсії); + 43
OHE-ознак -> разом 53
train: (160, 53) | test: (41, 53)
```

## Завдання 6. Побудова моделей класифікації

Класифікатор — **`LogisticRegression(max_iter=2000, class_weight='balanced')`** (функція `make_classifier`), параметри однакові для всіх варіантів.

- **Model A:** усі ознаки → Classifier
- **Model B:** відібрані ознаки → Classifier
- **Model C:** стандартизація → PCA → Classifier

Додатково (за бажанням): **C-90 %** (менше компонент) та **C-all** (PCA над *усіма* закодованими ознаками, 95 %).

```
pipelines = {
    'A': (Pipeline([('prep', make_preprocessor(numeric, categorical)), ('clf',
make_classifier())]), all_cols,
          'Усі ознаки'),
    'B': (Pipeline([('prep', make_preprocessor([c for c in numeric if c in set_B], [c for c in
categorical if c in set_B])),
                    ('clf', make_classifier())]), set_B, 'Відібрані ознаки'),
    'C': (Pipeline([('prep', make_preprocessor(numeric, categorical, pca_components=n95)),
('clf', make_classifier())]), all_cols,
          f'PCA ({n95} комп. замість {len(numeric)} числових)'),
    # додаткові варіанти
    'C-90%': (Pipeline([('prep', make_preprocessor(numeric, categorical, pca_components=n90)),
('clf', make_classifier())]), all_cols,
              f'PCA ({n90} комп., 90 %)'),
    'C-all': (Pipeline([('prep', make_preprocessor(numeric, categorical)), ('pca',
PCA(n_components=0.95, random_state=RANDOM_STATE)),
                        ('clf', make_classifier())]), all_cols, 'PCA над усіма закодованими
(95 %)'),
}
for k, (p, cols, _) in pipelines.items():
    p.fit(X_train[cols], y_train)         # fit ТІЛЬКИ на train
print('Навчено моделей:', list(pipelines))
```

```
Навчено моделей: ['A', 'B', 'C', 'C-90%', 'C-all']
```

# Завдання 7. Оцінювання моделей

Усі прогнози — на **одній і тій самій test-вибірці**.

```
results, preds, probas = [], {}, {}
for k, (p, cols, desc) in pipelines.items():
    pred = p.predict(X_test[cols]); proba = p.predict_proba(X_test[cols])[:, 1]
    preds[k], probas[k] = pred, proba
    n_out = p[:-1].transform(X_test[cols]).shape[1]
    results.append({'Варіант': k, 'Представлення даних': desc, 'Кількість ознак/компонент':
n_out,
                    'Accuracy': accuracy_score(y_test, pred), 'Precision':
precision_score(y_test, pred),
                    'Recall': recall_score(y_test, pred), 'F1': f1_score(y_test, pred),
                    'ROC-AUC': roc_auc_score(y_test, proba)})
results_df = pd.DataFrame(results).set_index('Варіант')
results_df.round(3)
```

| | Представлення даних | Кількість ознак/компонент | Accuracy | Precision | Recall | F1 | ROC-AUC |
|---|---|---|---|---|---|---|---|
| A | Усі ознаки | 60 | 0.976 | 0.875 | 1.000 | 0.933 | 0.996 |
| B | Відібрані ознаки | 36 | 0.951 | 0.778 | 1.000 | 0.875 | 0.987 |
| C | PCA (10 комп. замість 17 числових) | 53 | 0.927 | 0.700 | 1.000 | 0.824 | 0.987 |
| C-90% | PCA (7 комп., 90 %) | 50 | 0.951 | 0.778 | 1.000 | 0.875 | 0.992 |
| C-all | PCA над усіма закодованими (95 %) | 18 | 0.878 | 0.600 | 0.857 | 0.706 | 0.971 |

```
fig, axes = plt.subplots(1, len(pipelines), figsize=(4 * len(pipelines), 3.8))
for ax, k in zip(axes, pipelines):
    ConfusionMatrixDisplay(confusion_matrix(y_test, preds[k]), display_labels=['не дорогий',
'дорогий']).plot(ax=ax, cmap='Blues', colorbar=False)
    ax.set_title(f'Model {k}')
plt.tight_layout(); plt.show()


plt.figure(figsize=(6, 5))
for k in pipelines:
    fpr, tpr, _ = roc_curve(y_test, probas[k]); plt.plot(fpr, tpr, label=f'{k}
(AUC={roc_auc_score(y_test, probas[k]):.3f})')
plt.plot([0, 1], [0, 1], 'k--', alpha=.4); plt.xlabel('FPR'); plt.ylabel('TPR');
plt.title('ROC-криві (test)'); plt.legend(); plt.show()
```

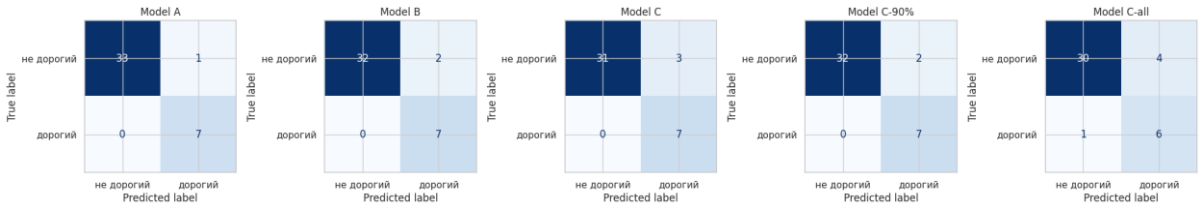



*Рисунок 6 — Матриці помилок (confusion matrix) для всіх моделей на test*

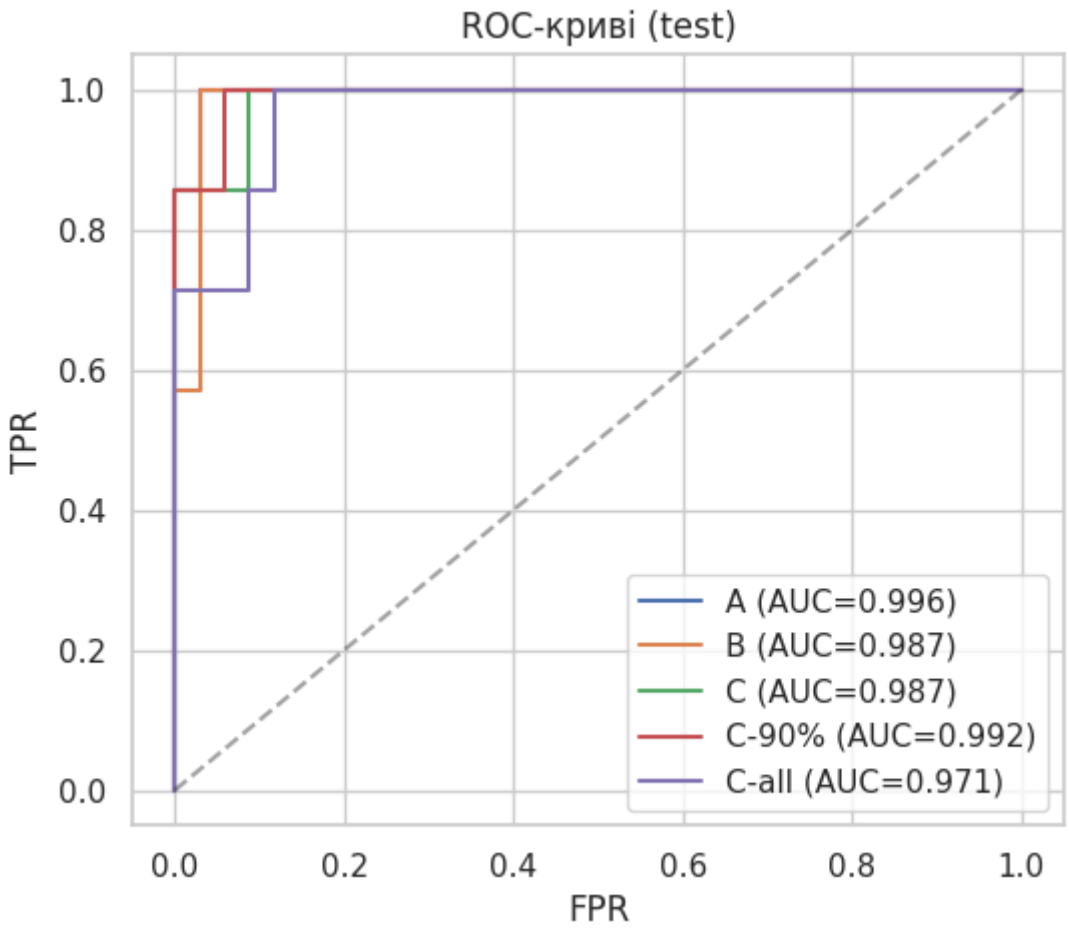



*Рисунок 7 — ROC-криві моделей на test*

***Додатково: стійкість результатів***

У test лише **41 об'єкт і ≈7 «дорогих»** — одна помилка зсуває Precision/Recall на 10–15 п.п. Тому, щоб висновки не трималися на одному випадковому розбитті, повторюємо порівняння за **повторною стратифікованою крос-валідацією (5×5) на train** (весь пайплайн, включно з PCA та імпутацією, перенавчається в кожному фолді — витоку немає). *Примітка:* список колонок набору B вибирався за train, тож для B ця крос-оцінка трохи оптимістична.

```
cv_rows = [cv_report(p, cols, k) for k, (p, cols, _) in pipelines.items()]
cv_df = pd.DataFrame(cv_rows)
cv_std = pd.DataFrame([{m: cross_validate(p, X_train[cols], y_train, cv=cv, scoring=scoring,
n_jobs=-1)[f'test_{m}'].std()
                       for m in scoring} for k, (p, cols, _) in pipelines.items()],
index=pipelines.keys())
display((cv_df.round(3).astype(str) + ' ± ' +
cv_std.round(3).astype(str)).rename_axis('Варіант'))
```

| | accuracy | precision | recall | f1 | roc_auc |
|---|---|---|---|---|---|
| A | 0.891 ± 0.057 | 0.679 ± 0.146 | 0.845 ± 0.144 | 0.742 ± 0.12 | 0.938 ± 0.052 |
| B | 0.882 ± 0.059 | 0.653 ± 0.146 | 0.875 ± 0.118 | 0.737 ± 0.111 | 0.952 ± 0.04 |
| C | 0.881 ± 0.062 | 0.657 ± 0.15 | 0.845 ± 0.144 | 0.726 ± 0.121 | 0.938 ± 0.052 |
| C-90% | 0.885 ± 0.061 | 0.666 ± 0.152 | 0.852 ± 0.139 | 0.735 ± 0.12 | 0.948 ± 0.047 |
| C-all | 0.868 ± 0.052 | 0.615 ± 0.115 | 0.831 ± 0.145 | 0.696 ± 0.101 | 0.933 ± 0.056 |

***Скільки ознак можна вилучити? Відбір за mutual information всередині Pipeline***

`SelectKBest(mutual_info_classif)` стоїть **усередині** пайплайна, тож для кожного фолда ранжування ознак рахується лише за його train-частиною.

```
ks = [3, 5, 8, 10, 15, 20, 30, 40, n_encoded]
rows = []
for k in ks:
    p = Pipeline([('prep', make_preprocessor(numeric, categorical)),
                  ('select', SelectKBest(lambda X, y: mutual_info_classif(X, y,
random_state=RANDOM_STATE), k=k)),
                  ('clf', make_classifier())])
    r = cross_validate(p, X_train[all_cols], y_train, cv=cv, scoring=scoring, n_jobs=-1)
    rows.append({'k (кодованих ознак)': k, 'Accuracy': r['test_accuracy'].mean(), 'F1':
r['test_f1'].mean(), 'ROC-AUC': r['test_roc_auc'].mean()})
sweep_k = pd.DataFrame(rows).set_index('k (кодованих ознак)')
display(sweep_k.round(3))


# аналогічно — кількість PCA-компонент (числовий блок)
rows = []
for k in [2, 3, 4, 5, 7, 10, 13, len(numeric)]:
    p = Pipeline([('prep', make_preprocessor(numeric, categorical, pca_components=k)), ('clf',
make_classifier())])
    r = cross_validate(p, X_train[all_cols], y_train, cv=cv, scoring=scoring, n_jobs=-1)
    rows.append({'PCA-компонент': k, 'поясн. дисперсія': cum[k-1], 'Accuracy':
r['test_accuracy'].mean(), 'F1': r['test_f1'].mean(), 'ROC-AUC': r['test_roc_auc'].mean()})
sweep_pca = pd.DataFrame(rows).set_index('PCA-компонент')
display(sweep_pca.round(3))


fig, ax = plt.subplots(1, 2, figsize=(12, 3.8))
sweep_k[['F1', 'ROC-AUC']].plot(marker='o', ax=ax[0]); ax[0].set_title('SelectKBest: якість vs
кількість ознак (CV на train)'); ax[0].set_xscale('log')
sweep_pca[['F1', 'ROC-AUC']].plot(marker='o', ax=ax[1]); ax[1].set_title('PCA: якість vs
кількість компонент (CV на train)')
plt.tight_layout(); plt.show()
```

| | Accuracy | F1 | ROC-AUC |
|---|---|---|---|
| 3 | 0.844 | 0.673 | 0.926 |
| 5 | 0.834 | 0.646 | 0.927 |
| 8 | 0.858 | 0.677 | 0.933 |
| 10 | 0.865 | 0.697 | 0.933 |
| 15 | 0.875 | 0.726 | 0.934 |
| 20 | 0.879 | 0.733 | 0.936 |
| 30 | 0.882 | 0.736 | 0.937 |
| 40 | 0.886 | 0.744 | 0.941 |
| 60 | 0.891 | 0.742 | 0.938 |

| | поясн. дисперсія | Accuracy | F1 | ROC-AUC |
|---|---|---|---|---|
| 2 | 0.609 | 0.886 | 0.735 | 0.950 |
| 3 | 0.693 | 0.882 | 0.730 | 0.949 |
| 4 | 0.757 | 0.889 | 0.739 | 0.947 |
| 5 | 0.816 | 0.882 | 0.732 | 0.950 |
| 7 | 0.901 | 0.885 | 0.735 | 0.948 |
| 10 | 0.966 | 0.881 | 0.726 | 0.938 |
| 13 | 0.990 | 0.890 | 0.740 | 0.937 |
| 17 | 1.000 | 0.891 | 0.742 | 0.938 |

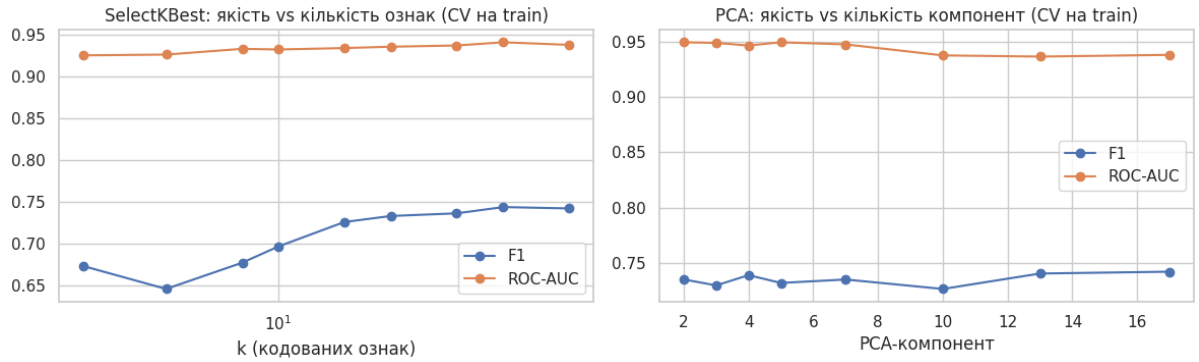



*Рисунок 8 — Залежність якості від кількості ознак (SelectKBest) та PCA-компонент (CV на train)*

# Завдання 8. Порівняльний аналіз

### *Зведена таблиця (test)*

```
final_table = results_df[['Представлення даних', 'Кількість ознак/компонент', 'Accuracy',
'Precision', 'Recall', 'F1', 'ROC-AUC']].round(3)
final_table
```

| | Представлення даних | Кількість ознак/компонент | Accuracy | Precision | Recall | F1 | ROC-AUC |
|---|---|---|---|---|---|---|---|
| A | Усі ознаки | 60 | 0.976 | 0.875 | 1.000 | 0.933 | 0.996 |
| B | Відібрані ознаки | 36 | 0.951 | 0.778 | 1.000 | 0.875 | 0.987 |
| C | PCA (10 комп. замість 17 числових) | 53 | 0.927 | 0.700 | 1.000 | 0.824 | 0.987 |
| C-90% | PCA (7 комп., 90 %) | 50 | 0.951 | 0.778 | 1.000 | 0.875 | 0.992 |
| C-all | PCA над усіма закодованими (95 %) | 18 | 0.878 | 0.600 | 0.857 | 0.706 | 0.971 |

### *Результати на test (41 об'єкт, 7 «дорогих») та їх стійкість*

| Варіант | Ознак (кодов.) | Accuracy | F1 | ROC-AUC | Помилки на test | CV на train: F1 / ROC-AUC |
|---|---|---|---|---|---|---|
| A — усі | 60 | 0.976 | 0.933 | 0.996 | 1 FP | 0.742 / 0.938 |
| B — відібрані | 36 (11 колонок) | 0.951 | 0.875 | 0.987 | 2 FP | 0.737 / 0.952 |
| C — PCA (10 комп.) | 53 | 0.927 | 0.824 | 0.987 | 3 FP | 0.726 / 0.938 |
| C-90 % (7 комп.) | 50 | 0.951 | 0.875 | 0.992 | 2 FP | 0.735 / 0.948 |
| C-all (PCA над усім) | 18 | 0.878 | 0.706 | 0.971 | 1 FN + 4 FP | 0.696 / 0.933 |

***Важливе застереження.*** *Різниця між A, B і C на test — це 1–2 помилкові об'єкти з 41. Крос-валідація (5×5) показує, що A, B і C лежать у межах одного стандартного відхилення (σ(F1) ≈ 0.11–0.12). Тож чесний висновок — «суттєвої різниці немає», а не «A краще».*

**1. Чи покращив відбір ознак якість?** Ні. На test B трохи гірший за A (F1 0.875 проти 0.933 — одна додаткова хибнопозитивна помилка), у CV — F1 однаковий (0.737 проти 0.742), а ROC-AUC навіть дещо вищий (0.952 проти 0.938, але в межах σ ≈ 0.04–0.05). Recall = 1.0 в обох на test. Відбір **не покращив**, але й **не погіршив** якість.

**2. Як змінилась точність після зменшення кількості ознак?** Accuracy на test упала з 0.976 до 0.951 (−2.5 п.п. = один об'єкт); у CV — з 0.891 до 0.882 (−0.9 п.п.). Тобто практично не змінилась.

**3. Скільки ознак можна вилучити без суттєвого погіршення?** Набір B прибирає **15 з 26 колонок (≈58 %)** з незмінною якістю в CV. За ранжуванням mutual information: зі 60 кодованих ознак можна залишити **≈20** (F1 0.733 проти 0.742 для 60; ROC-AUC 0.936 проти 0.938) — це −2/3 ознак; при 10 ознаках F1 падає до 0.70, при 5 — до 0.65. Тож межа — приблизно 15–20 кодованих ознак.

**4. Яку частку дисперсії пояснюють PCA-компоненти?** 10 компонент — **96.6 %** (поріг 95 %); 7 компонент — 90.1 %; перша компонента — 43.4 %.

**5. Як PCA вплинув на Accuracy, F1, ROC-AUC?** На test PCA (C) трохи погіршив: Accuracy 0.976 → 0.927, F1 0.933 → 0.824, ROC-AUC 0.996 → 0.987 (Recall лишився 1.0, зросла кількість хибнопозитивних: 1 → 3). Із 7 компонентами (C-90 %) результат кращий (F1 0.875, AUC 0.992). У CV різниця зникає: F1 0.726 проти 0.742, ROC-AUC однаковий (0.938). Варіант C-all (PCA над усіма закодованими ознаками разом із OHE) **помітно гірший** (F1 0.706 на test, 0.696 у CV): стиснення 60 ознак до 18 змішує числові й бінарні ознаки та «розмиває» сильний сигнал від `make`.

**6. Чи виправдане використання PCA?** Для цього набору — **ні**. По-перше, PCA не покращив якості. По-друге, він стискає лише числовий блок (17 → 10), а загальна розмірність зменшується всього з 60 до 53 ознак. По-третє, втрачається інтерпретованість (компоненти — суміші ознак). Єдиний аргумент «за» — числові ознаки справді сильно колінеарні (PC1 = 43 %), тож PCA — наочна ілюстрація цієї надлишковості; для логістичної регресії з L2-регуляризацією вона й так не критична. Показово, що в CV достатньо навіть 2–3 компонент (F1 ≈ 0.73, ROC-AUC ≈ 0.95): основну інформацію несе один фактор «розмір/потужність» плюс марка.

**7. Найкраще співвідношення «якість / складність»? Набір B** (11 колонок, 36 кодованих ознак): якість у CV така ж, як у A (F1 0.737 vs 0.742, AUC 0.952 vs 0.938), у 2.4 раза менше колонок (11 проти 26), легше інтерпретувати й підтримувати, менше ризик перенавчання. Набір A дає трохи кращий результат на test (різниця в 1 об'єкт), але вимагає всіх 26 колонок. PCA-варіанти складніші за B і не кращі.

## Завдання 9. Фінальний pipeline

```
Raw Data → Cleaning → Train/Test Split → Preprocessing → Feature Selection → Classifier → Prediction → Evaluation
```

На основі експериментів обираємо **Набір B + стандартизація/OHE + `LogisticRegression(class_weight='balanced')`** — найкращий компроміс між якістю та складністю (Завдання 8, п. 7). Збираємо **один об'єкт `Pipeline`**, який приймає **сирі дані** (із символами `?`, помилками в назвах марок тощо) і сам виконує очищення — це гарантує, що на етапі прогнозу над новими даними виконуються рівно ті самі кроки.

```
from sklearn.base import clone

# ---- 1. Raw Data
raw = pd.read_csv('Automobile_data.csv')
price_num = pd.to_numeric(raw['price'].replace('?', np.nan))
has_price = price_num.notna()                                    # рядки без таргета не придатні для навчання/оцінки
y_raw = (price_num[has_price] > PRICE_THRESHOLD).astype(int).reset_index(drop=True)
X_raw = raw.loc[has_price].drop(columns='price').reset_index(drop=True)   # price прибрано (витік)

# ---- 2. Cleaning: безстанові перетворення (нічого не «вчаться» з даних) -> безпечно виносити в Pipeline
def clean_features(d):
    d = d.copy().replace('?', np.nan)
    for c in ['normalized-losses', 'bore', 'stroke', 'horsepower', 'peak-rpm']:
        d[c] = pd.to_numeric(d[c])
    d['make'] = d['make'].replace({'peugot': 'peugeot', 'alfa-romero': 'alfa-romeo'})
    d['n1_missing'] = d['normalized-losses'].isna().astype(int)
    d['num-of-cylinders'] = d['num-of-cylinders'].map(cyl_map)
    return d

# ---- 3. Train/Test split (стратифікований) — ДО будь-якого «навчання»
Xr_train, Xr_test, yr_train, yr_test = train_test_split(X_raw, y_raw, test_size=0.2, stratify=y_raw, random_state=RANDOM_STATE)

# ---- 4-6. Preprocessing -> Feature Selection (набір B) -> Classifier
final_pipe = Pipeline([
    ('cleaning', FunctionTransformer(clean_features)),
    ('preprocessing', make_preprocessor([c for c in numeric if c in set_B],      # імпутація + StandardScaler
                                        [c for c in categorical if c in set_B])), # імпутація + OneHot; інші колонки відкидаються
    ('classifier', make_classifier()),
])
final_pipe.fit(Xr_train, yr_train)                               # усе навчається тільки на train

# ---- 7-8. Prediction -> Evaluation
pred = final_pipe.predict(Xr_test); proba = final_pipe.predict_proba(Xr_test)[:, 1]
final_metrics = pd.Series({'Accuracy': accuracy_score(yr_test, pred), 'Precision': precision_score(yr_test, pred),
                           'Recall': recall_score(yr_test, pred), 'F1': f1_score(yr_test, pred),
                           'ROC-AUC': roc_auc_score(yr_test, proba)}, name='Final pipeline (B)')
display(final_metrics.round(3).to_frame().T)
print('Confusion matrix [[TN FP], [FN TP]]:', confusion_matrix(yr_test, pred).tolist())
print('Збігається з Model B із Завдання 7:', np.allclose(final_metrics.values, results_df.loc['B', final_metrics.index].values))

# прогноз для «нового» автомобіля прямо з сирого рядка
new_cars = Xr_test.head(3)
display(pd.DataFrame({'make': new_cars['make'].values, 'horsepower': new_cars['horsepower'].values,
                      'P(дорогий)': final_pipe.predict_proba(new_cars)[:, 1].round(3),
                      'прогноз': final_pipe.predict(new_cars), 'факт': yr_test.head(3).values}))
final_pipe
```

| | Accuracy | Precision | Recall | F1 | ROC-AUC |
|---|---|---|---|---|---|
| Final pipeline (B) | 0.951 | 0.778 | 1.0 | 0.875 | 0.987 |

```
Confusion matrix [[TN FP], [FN TP]]: [[32, 2], [0, 7]]
Збігається з Model B із Завдання 7: True
```

| | make | horsepower | P(дорогий) | прогноз | факт |
|---|---|---|---|---|---|
| 0 | mitsubishi | 116 | 0.023 | 0 | 0 |
| 1 | volvo | 114 | 0.852 | 1 | 0 |
| 2 | saab | 110 | 0.297 | 0 | 0 |

### *Які етапи навчаються тільки на train і чому це важливо*

| Етап | Що «вчиться» | Тільки на train? |
|---|---|---|
| Cleaning (`?`→NaN, виправлення назв, мапінг циліндрів) | нічого — фіксовані правила | не потрібно (безстановий) |
| Видалення рядків без `price`, створення таргета за порогом 18 000 | нічого — фіксований бізнес-поріг | не потрібно |
| Імпутація (медіани, моди) | значення медіан/мод | **так** |
| `StandardScaler` | середнє й стандартне відхилення | **так** |
| `OneHotEncoder` | словник категорій, групування рідкісних | **так** |
| Відбір ознак (кореляція з таргетом, Крамер V, мультиколінеарність, MI) | порогові статистики за таргетом | **так** (у роботі — за `X_train`/`y_train`) |
| PCA | головні компоненти | **так** |
| `class_weight='balanced'` | ваги класів із `y_train` | **так** |
| Класифікатор | коефіцієнти | **так** |

**Чому це необхідно.** Якщо, наприклад, масштабувати, імпутувати чи відбирати ознаки на *всіх* даних, статистики test-вибірки (її середнє, медіана, кореляції з таргетом) потрапляють у навчання. Модель побачить «відбиток» test ще до оцінювання, і метрики стануть завищеними — на справді нових даних якість виявиться нижчою (**data leakage**). Особливо небезпечний відбір ознак за кореляцією з таргетом на всьому наборі: ознаки, які випадково «збіглися» з таргетом у test, потрапили б у модель. Pipeline гарантує, що `fit` викликається лише на train, а для test застосовується тільки `transform`/`predict`; у крос-валідації весь пайплайн перенавчається в кожному фолді.

**Залишковий ризик.** Через «близнюків» (різні комплектації однієї моделі, 14 рядків) схожі автомобілі можуть опинитися і в train, і в test — це легкий оптимізм оцінки, який при 201 записі повністю усунути неможливо; суворіше було б ділити за групами (`make`), але тоді цілі марки люкс-класу випали б із навчання.

## Висновки

- **Кількість ознак.** Початково — **26 колонок** у `X` (25 вихідних без `price` + індикатор `nl_missing`; 17 числових і 9 категоріальних), після кодування — **60 ознак**. У фінальному варіанті (набір B) — **11 колонок** (7 числових + 4 категоріальні), що після кодування дає **36 ознак**. У варіанті PCA 17 числових ознак замінено **10 головними компонентами** (96.6 % дисперсії), разом 53 ознаки.
- **Порівняння представлень.** На test: A (усі) — F1 = 0.933, ROC-AUC = 0.996; B (відібрані) — 0.875 / 0.987; C (PCA) — 0.824 / 0.987. Різниця — 1–2 об'єкти з 41, тому за повторною крос-валідацією (5×5) усі три варіанти статистично нерозрізнювані (F1 ≈ 0.73–0.74, ROC-AUC ≈ 0.94–0.95). PCA над усіма закодованими ознаками (C-all) гірший за всіх (F1 = 0.71).
- **Найкращий варіант.** За якістю — A і B практично рівні; за співвідношенням «якість / складність» — **набір B**: ту саму якість забезпечують 11 колонок замість 26.
- **Вплив інженерії ознак.** Найважливіше — правильне очищення (`?` → NaN, типи, виправлення назв), коректне масштабування/кодування в `Pipeline` і врахування дисбалансу (`class_weight` підняв Recall «дорогих» авто з 0.70 до 0.85 у CV). Відбір ознак не підвищив метрик, але вдвічі спростив модель без втрат: майже вся інформація зосереджена у кількох ознаках (потужність/об'єм двигуна, економічність, привід, марка).
- **Доцільність PCA.** Для цього набору **не виправдана**: ознаки в числовому блоці колінеарні (PC1 = 43 %), але PCA не покращив метрик, стиснув загальну розмірність лише з 60 до 53 і зробив ознаки неінтерпретованими. Варіант з PCA над усіма закодованими ознаками погіршив результат. PCA корисний як діагностика мультиколінеарності.
- **Обґрунтування фінального pipeline.** `Cleaning → Train/Test split → Imputer + StandardScaler / OneHot → вибір 11 колонок (набір B) → LogisticRegression(class_weight='balanced')`. Усі навчальні кроки виконуються лише на train, очищення — безстанове, тому модель приймає сирі дані й не має витоку. Він відтворює результати Model B (Accuracy 0.951, Recall 1.0, F1 0.875, ROC-AUC 0.987).
- **Обмеження.** Вибірка мала (41 об'єкт у test, 7 у міноритарному класі), тому метрики на test нестабільні; клас «дорогий» визначено порогом 18 000 $ (інший поріг дасть інші числа); задача відносно легка (ROC-AUC ≈ 0.94–0.99).